Multi-AI — Project Integration & Architecture Report

Table of Contents

Chapter 1: Introduction

1.1 Problem Statement

In the rapidly evolving domain of Artificial Intelligence, users face a fragmented ecosystem when querying Large Language Models (LLMs). Different providers (Google, OpenAI, Anthropic, Groq, Cohere) deliver responses varying in tone, accuracy, response speed, and formatting. Developers and researchers are forced to manually open multiple browser tabs or write ad-hoc API scripts to compare outputs, which is inefficient, time-consuming, and makes real-time benchmark analysis impossible.

1.2 Existing System

The existing system relies on manual switching between separate web consoles (e.g., ChatGPT web console, Gemini chat interface, Anthropic console). There is no native way to send a single query to multiple models concurrently, nor is there any backend pipeline to automatically evaluate, synthesize, or compare their outputs side-by-side on a single dashboard layout.

1.3 Proposed System

The proposed Multi-AI system introduces a unified, responsive hub that coordinates connection streams to multiple AI services simultaneously. It features:

- A single prompt broadcast dock that fires concurrent API queries.
- Side-by-side comparison cards with individual token-count telemetries.
- Autonomous AI evaluation (Best-Response judge) and multi-response consensus synthesis.
- Collaborative AI-to-AI debate simulators.
- Mobile and desktop synchronized preference trackers, scratchpads, and custom personas.

1.4 Scope

Multi-AI provides a local and production-ready environment that can be deployed on servers (e.g., Render, Docker containers). It is designed to assist software engineers, researchers, and prompt-architects in optimizing comparative prompting workflows.

1.5 Objectives

- Minimize the time required to compare multiple AI models from minutes to seconds.
- Provide automated consensus and judge summaries to extract objective truths from varied model responses.
- Implement responsive dark/light styling and state management variables that sync across all device form-factors.

Chapter 2: System Requirement Specifications

2.1 Software Requirements

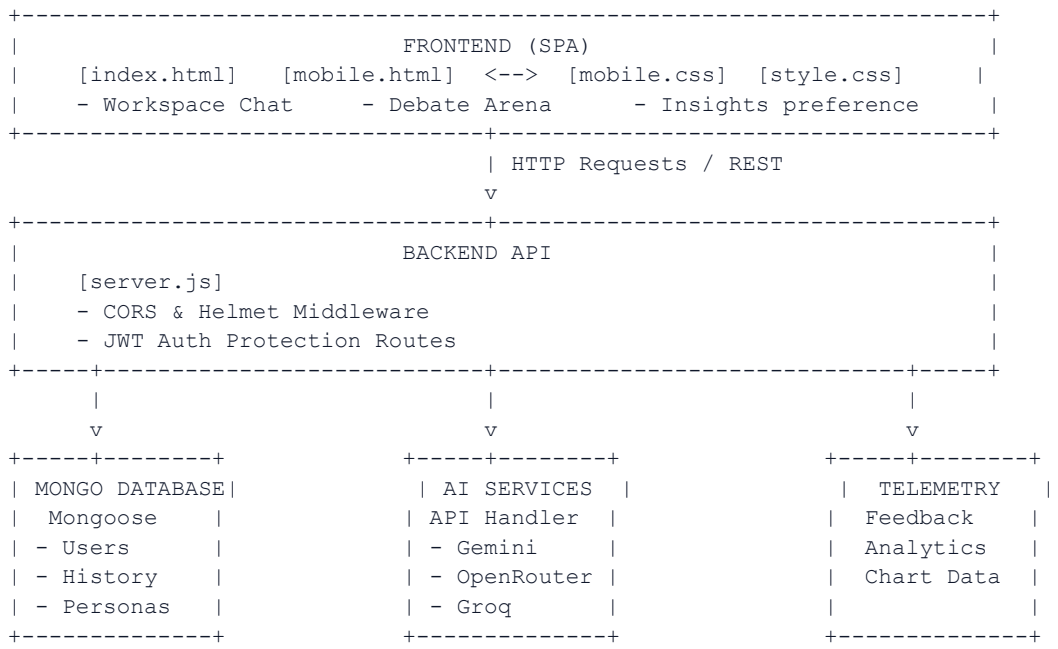
The project uses standard modern open-source stacks:

- **Operating System**: Windows / Linux / macOS compatible.
- **Runtime Environment**: Node.js v20.x or higher.
- **Web Server Framework**: Express.js v4.x.
- **Database Management**: MongoDB Community Server v6.x / MongoDB Atlas.
- **Database Object Modeling**: Mongoose ODM v9.x.
- **Frontend Core**: HTML5, Vanilla CSS3, Javascript (ES6+).
- **Visualization Library**: Chart.js v4.x.

2.2 Hardware Requirements

- **Processor**: Intel Core i5 or higher (or equivalent AMD/Apple Silicon).
- **Memory**: 8 GB RAM minimum (16 GB recommended).
- **Storage**: 500 MB of available hard-drive space.
- **Network**: Broadband internet connection for external LLM API fetches.

2.3 System Architecture



Chapter 3: Design & Implementation

3.1 Features List

- **Multi-Model workspace**: Active grid displaying responses with individual copy, download, like, and dislike actions.
- **Speech-to-Text Integration**: Native voice recognition triggers speech capture.
- **Prompt Optimizer**: An automated service that refines user queries into detailed prompts.
- **AI Consensus Synthesis**: Interrogates multiple responses and constructs a unified consensus.
- **Automated Judge (Best-Response)**: Assesses responses and selects the highest-scoring candidate.
- **AI Debate Arena**: Multi-round discussions between selected models.
- **Floating Scratchpad**: Text area supporting drag, minimization, local auto-saves, and backups.

3.2 Modules Description

- **Authentication**: JWT token verification protecting user preference saves.
- **AI Controller**: Orchestrates queries to Gemini API, OpenRouter (Liquid LFM), HuggingFace (Qwen), Groq (Llama), and Cohere.
- **Feedback Services**: Registers likes/dislikes in MongoDB, generating stats for Chart.js.
- **Workspace Settings**: Manages custom accent colors, secure BYOK (Bring Your Own Key) entries, and profile initials.

3.3 UML Diagrams

```
User Actions Sequence:
 User -> Input Prompt -> Clicks Send -> Frontend broadcasts request
   Frontend -> REST Route /api/ai/chat -> Backend processes payload
     Backend -> Parallel API Requests -> Gemini, Liquid, Groq Services
     Backend <- Responses Returned <- API Services
   Frontend <- JSON Output <- Backend
 User <- Visual Typewriter Stream <- Frontend renders cards
```

3.4 Environmental Setup

Create a `.env` configuration file in the backend root directory:

```
PORT=5000
NODE_ENV=production
MONGODB_URI=mongodb://127.0.0.1:27017/multi-ai
JWT_SECRET=super-secure-token-key-change-me
GEMINI_API_KEY=YOUR_GEMINI_KEY
OPENROUTER_API_KEY=YOUR_OPENROUTER_KEY
GROQ_API_KEY=YOUR_GROQ_KEY
```

3.5 Implementation & Methodology Steps

1. **API Integration**: Establish parallel async HTTP fetch calls using Axios and configure timeout parameters.

2. **State Design**: Map localStorage keys (`themeName`, `byok-keys`, `workspaceScratchpadText`) to automatically save UI states.
3. **Database Bulk Operations**: Utilize MongoDB bulkWrite for debounced conversation history saves.

Chapter 4: Results & Discussion

4.1 UI Screenshots

Figure 1: Sign-In Portal

The central login screen provides secure credentials submission and an instant guest bypass link.

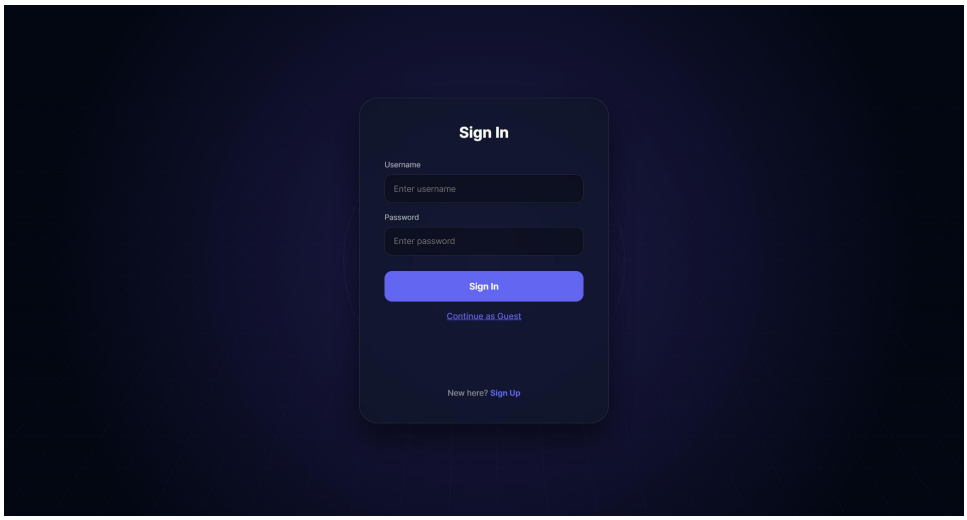


Figure: Sign-In Portal

Figure 2: Workspace Landing Panel

The desktop workspace viewport displaying active AI cards, sidebar selectors, and the central prompt bar.

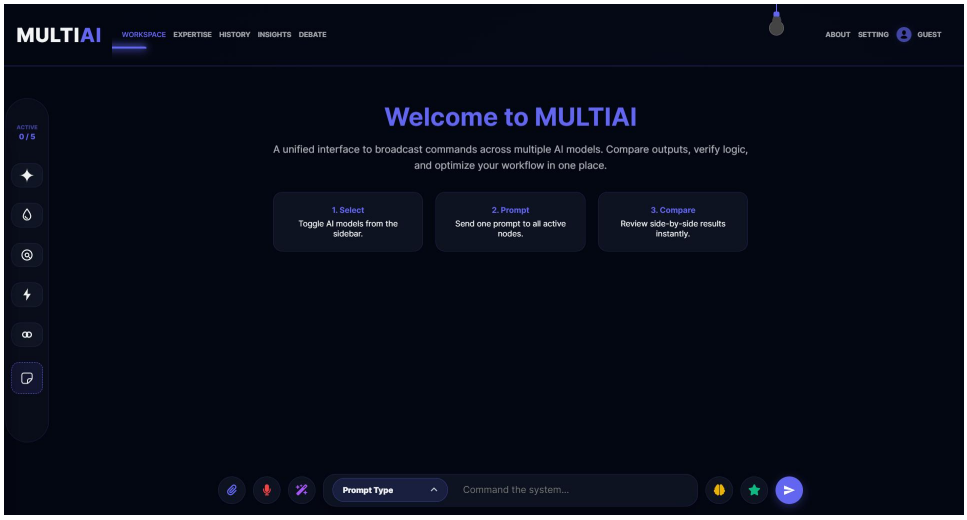


Figure: Workspace Landing Panel

Figure 3: Multi-Model Query Output

Side-by-side prompt responses featuring typewriter output streams and real-time telemetry token badges.

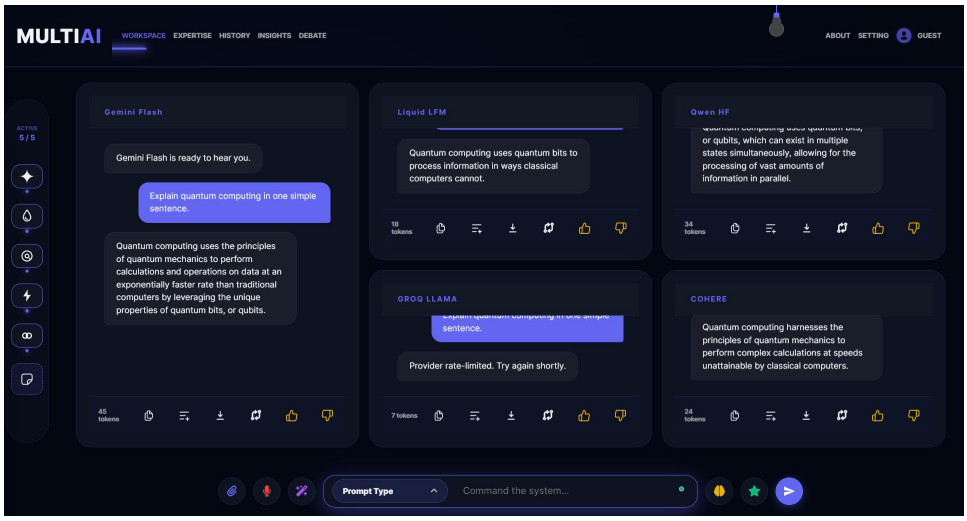


Figure: Multi-Model Query Output

Figure 4: Model Expertise Customizer

Allows selecting active system instructions to steer model behavior.

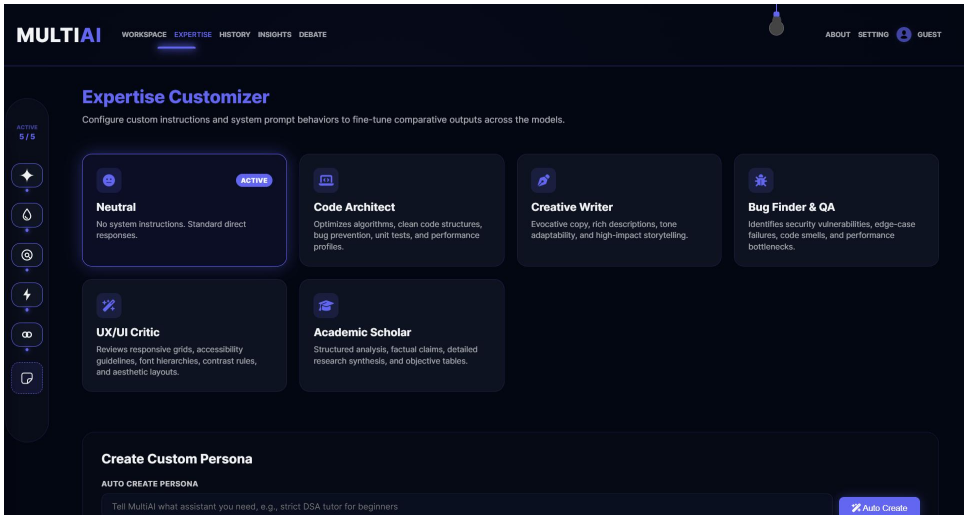


Figure: Model Expertise Customizer

Figure 5: Session History Log

Displays and restores past user prompts, responses, and settings.

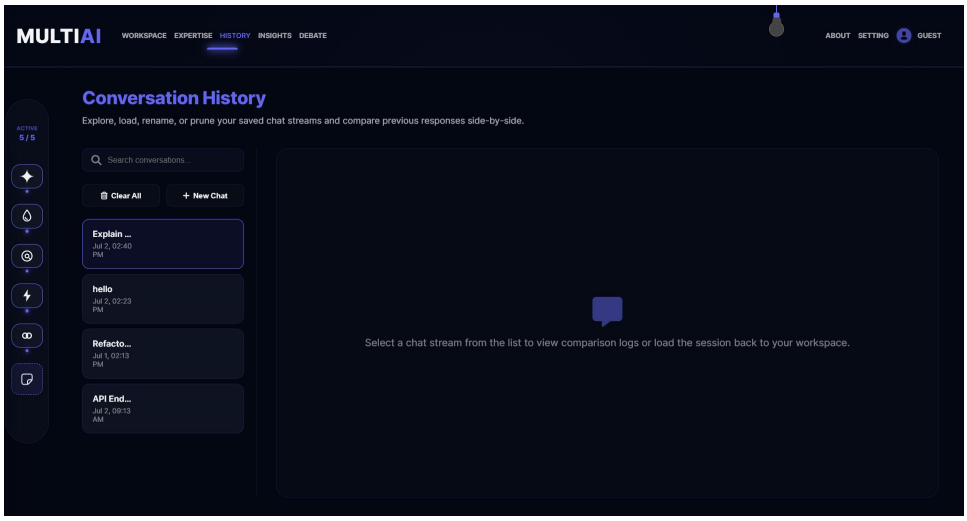


Figure: Session History Log

Figure 6: Workspace Analytics Dashboard

Uses Chart.js to render feedback stats (likes and dislikes) for each model.

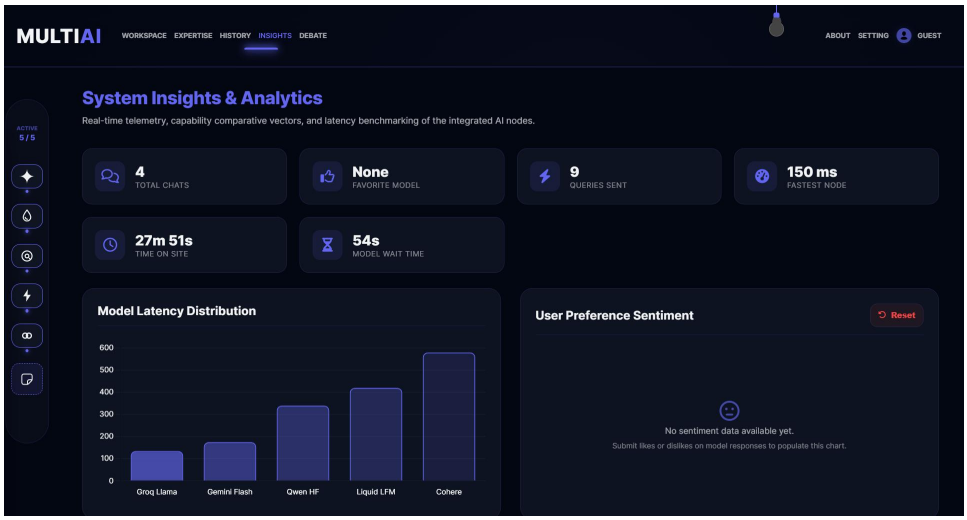


Figure: Workspace Analytics Dashboard

Figure 7: Automated AI Debate Arena

Simulates structured debates between two different LLM models.

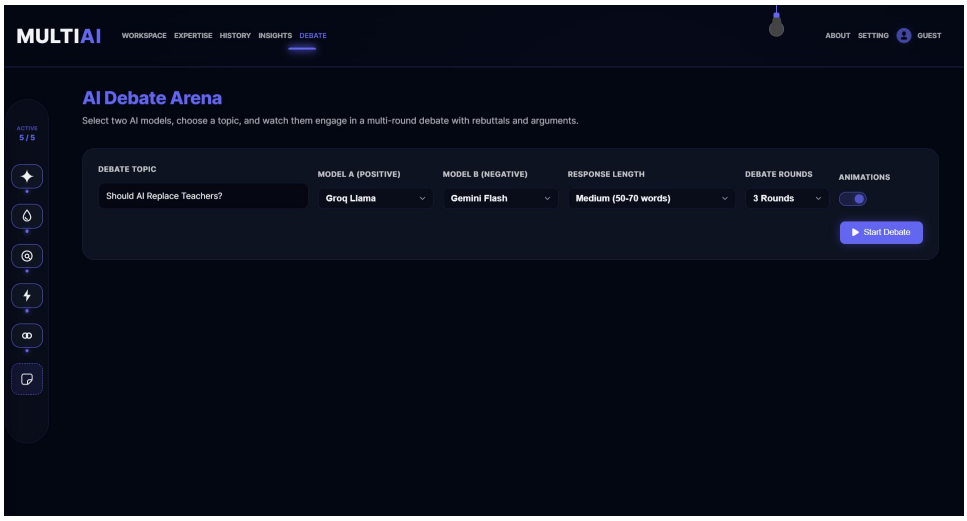


Figure: Automated AI Debate Arena

Figure 8: Secure BYOK Settings Panel

Allows entering custom API keys securely stored in the browser.

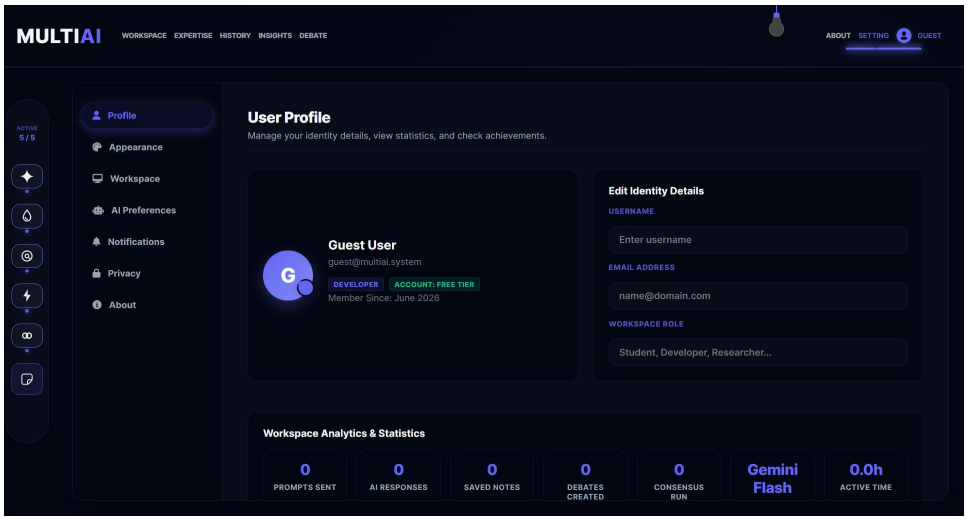


Figure: Secure BYOK Settings Panel

Figure 9: Mobile Workspace View

The mobile interface features a horizontal swiper menu and compact bottom action dock.

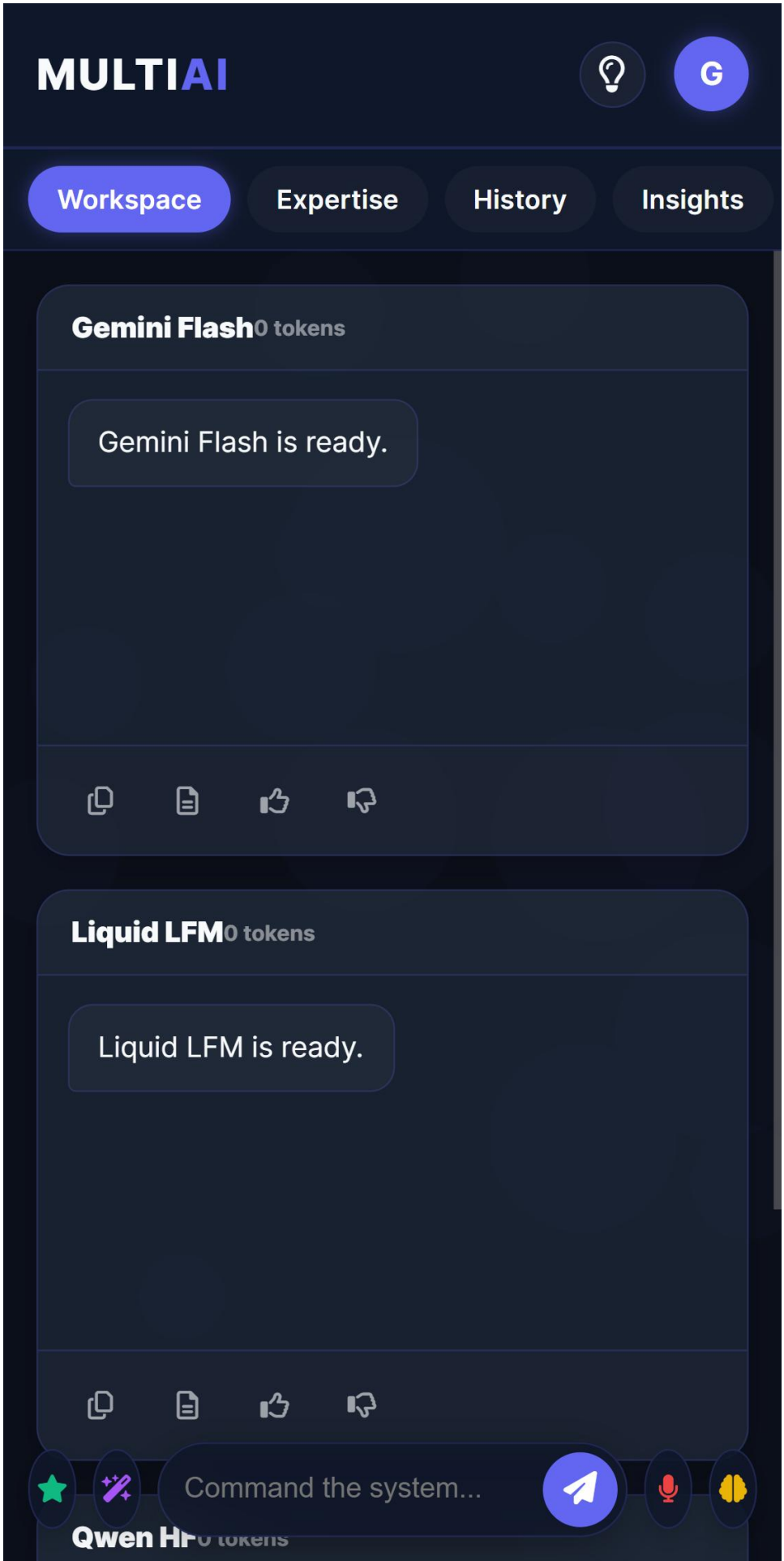

MULTIAI
G
Workspace
Expertise
History
Insights
Gemini Flash0 tokens
Gemini Flash is ready.
Liquid LFM0 tokens
Liquid LFM is ready.
Command the system...

Figure: Mobile Workspace View

Figure 10: Mobile Response Output

Shows comparative responses rendering correctly on mobile screen viewports.

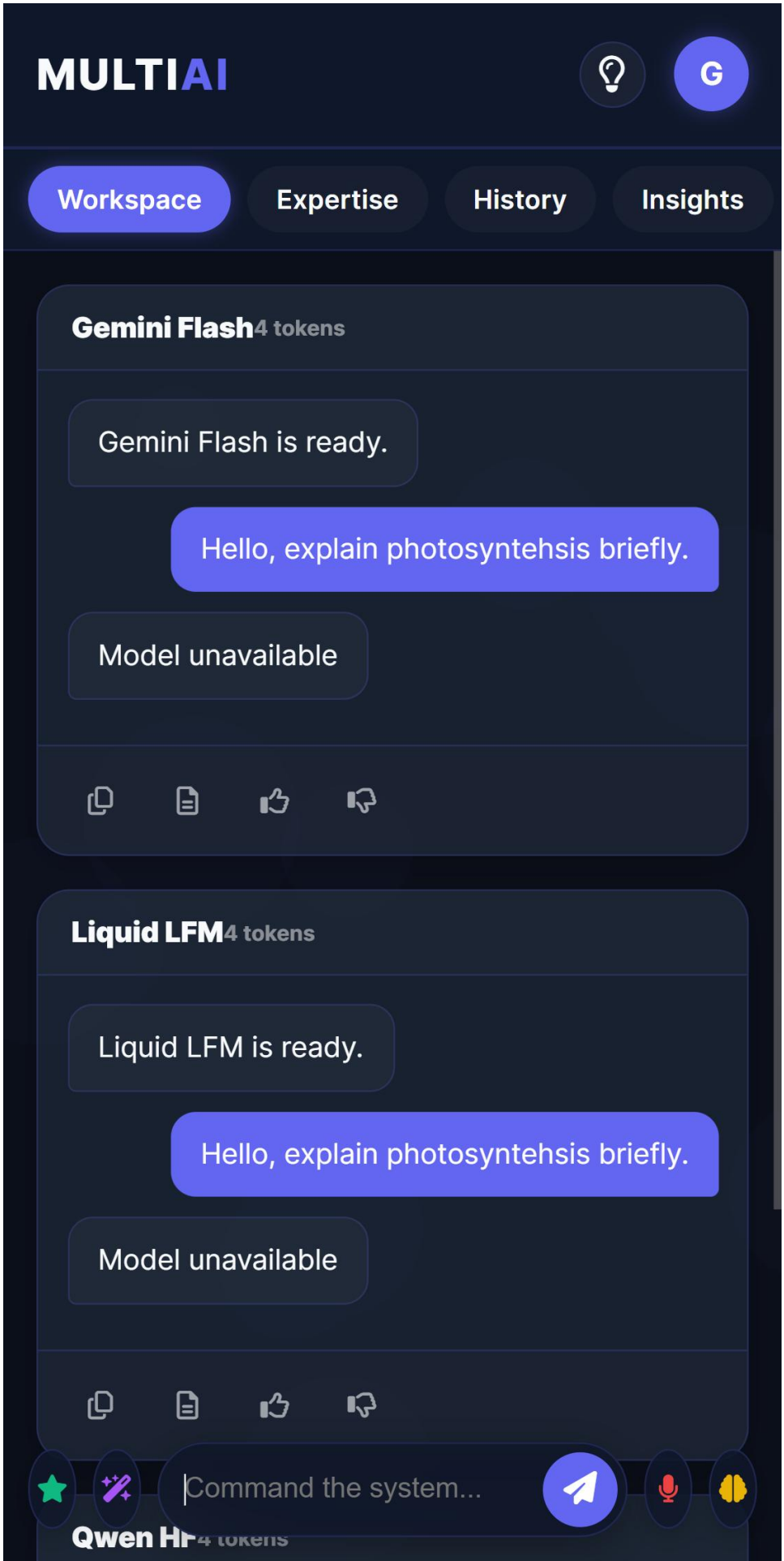

MULTIAI
G
Workspace
Expertise
History
Insights
Gemini Flash4 tokens
Gemini Flash is ready.
Hello, explain photosyntehsis briefly.
Model unavailable
Liquid LFM4 tokens
Liquid LFM is ready.
Hello, explain photosyntehsis briefly.
Model unavailable
Command the system...
Qwen Hr4 tokens

Figure: Mobile Response Output

4.2 Validation & Testing

- **API Tests**: Validated that custom keys overwrite system defaults.
- **Responsive Layout Tests**: Simulated mobile viewports and confirmed grid flattening behaves correctly.
- **MongoDB Tests**: Verified that saving history does not corrupt existing entries.

Chapter 5: Conclusion & Future Enhancements

5.1 Conclusion

The Multi-AI integration platform successfully centralizes, coordinates, and contrasts LLM responses on both mobile and desktop platforms. By automating consensus synthesis and response scoring, it streamlines prompt verification workflows.

5.2 Future Enhancements

- **Streaming Responses**: Enable Server-Sent Events (SSE) for word-by-word streaming in the workspace cards.
- **Offline LLM Integration**: Incorporate local WebGPU WebLLM runs directly inside the client browser.

References

4. Fielding, R. "Architectural Styles and the Design of Network-based Software Architectures", 2000.
5. MDN Web Docs. "Using the Speech Recognition API", 2026.
6. Chart.js Documentation. "Bar Chart Configuration Guide", 2026.

Appendix: Source/Pseudo Code

```
// Dynamic Prompt Broadcast Flow
async function broadcast() {
    const input = document.getElementById('main-in');
    const prompt = input.value.trim();
    if (!prompt) return;

    const activeTargets = getActiveWorkspaceModels();

    // Fire REST request
    const response = await fetch('/api/ai/chat', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            prompt: prompt,
            models: activeTargets.map(m => m.key)
        })
    });

    const data = await response.json();
    updateUIWorkspaceCards(data.responses);
}
```

List of Abbreviations

- **HTML**: HyperText Markup Language
- **CSS**: Cascading Style Sheets
- **JS**: JavaScript
- **API**: Application Programming Interface
- **NoSQL**: Not Only SQL (Database type)
- **JWT**: JSON Web Token
- **UI/UX**: User Interface / User Experience